

C# and ASP.NET

Practical Journal

UID : 2405010

Roll No : 12

Name : Yashoda Bhandari

Academic Year : 2026-27

Class : TY Bsc IT

Index

Practical No 01: Introduction to C# Programming – Part 1	Date: 15/06/2026
Practical No 02: Introduction to C# Programming – Part 2	Date: 16/06/2026
Practical No 03: Basic LINQ implementation in C#	Date: 17/06/2026
Practical No 04: Date and Time in LINQ implementation in C#	Date: 25/06/2026
Practical No 05: Inheritance and hierarchy – Part 1	Date: 02/07/2026
Practical No 06: Inheritance and hierarchy – Part 2	Date: 02/07/2026
Practical No 07: Structures in C#	Date: 09/07/2026
Practical No 08: ASP.NET Razor Pages	Date: 09/07/2026

Practical No : 01

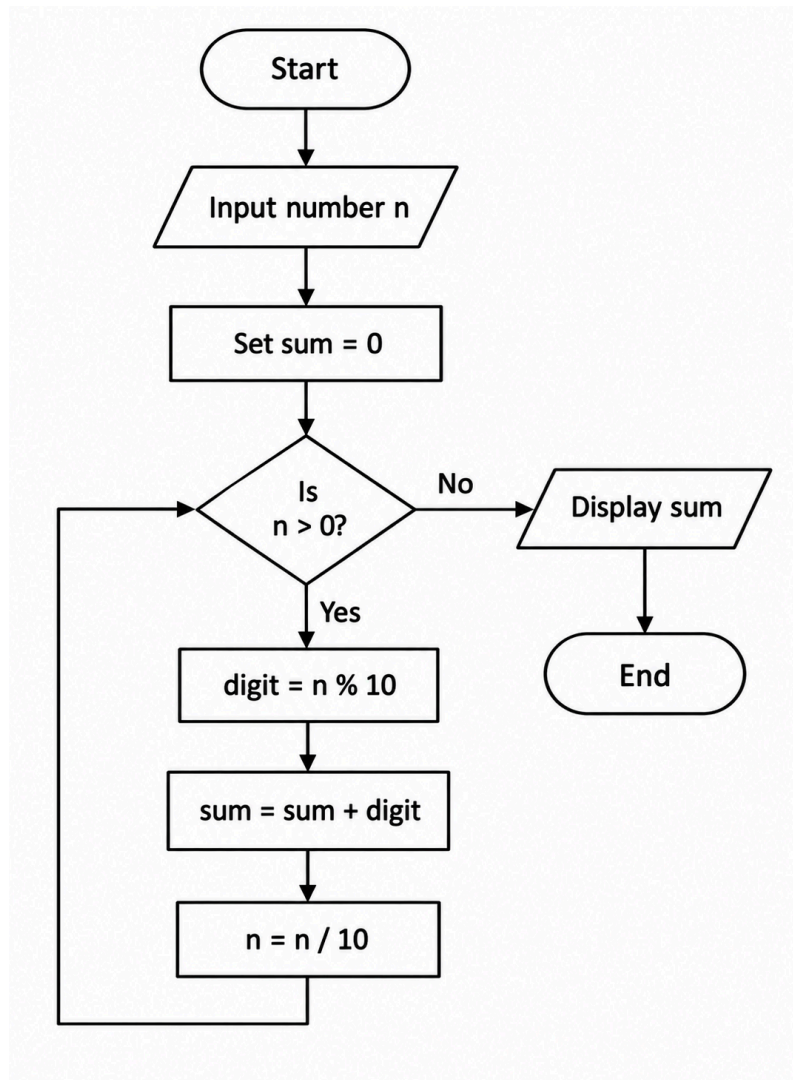
Q1. Write a C# code to find the sum of digits.

Title : Introduction to C# Programming – Part 1

Date : 15/06/2026

Aim: To demonstrate use of variables, loops, conditional statements and functions in C#.

Flow Diagram / Logic:



The program reads an integer from the user, then repeatedly extracts the last digit using the modulus operator ($\% 10$), adds it to a running total, and removes the last digit by integer division ($/ 10$). This continues in a while loop until the number becomes 0, at which point the accumulated total is the sum of digits.

Program:

```
namespace TYIT_Rep01
{
    internal class Program
    {
        static void Main()
        {
            Console.WriteLine("Enter a number : ");
            int n = int.Parse(Console.ReadLine());
            int sum = 0;
            int m = 0;
            while (n > 0)
            {
                m = n % 10;
                sum += m;
                n = n / 10;
            }
            Console.WriteLine(" Sum = " + sum);
        }
    }
}
```

Output:

```
Enter a number :
877437
Sum = 36

C:\Users\SXC_Student\...\TYIT_Rep01.exe (process 15344) exited with code 0 (0x0).
Press any key to close this window . . .
```

Error and Debug : None

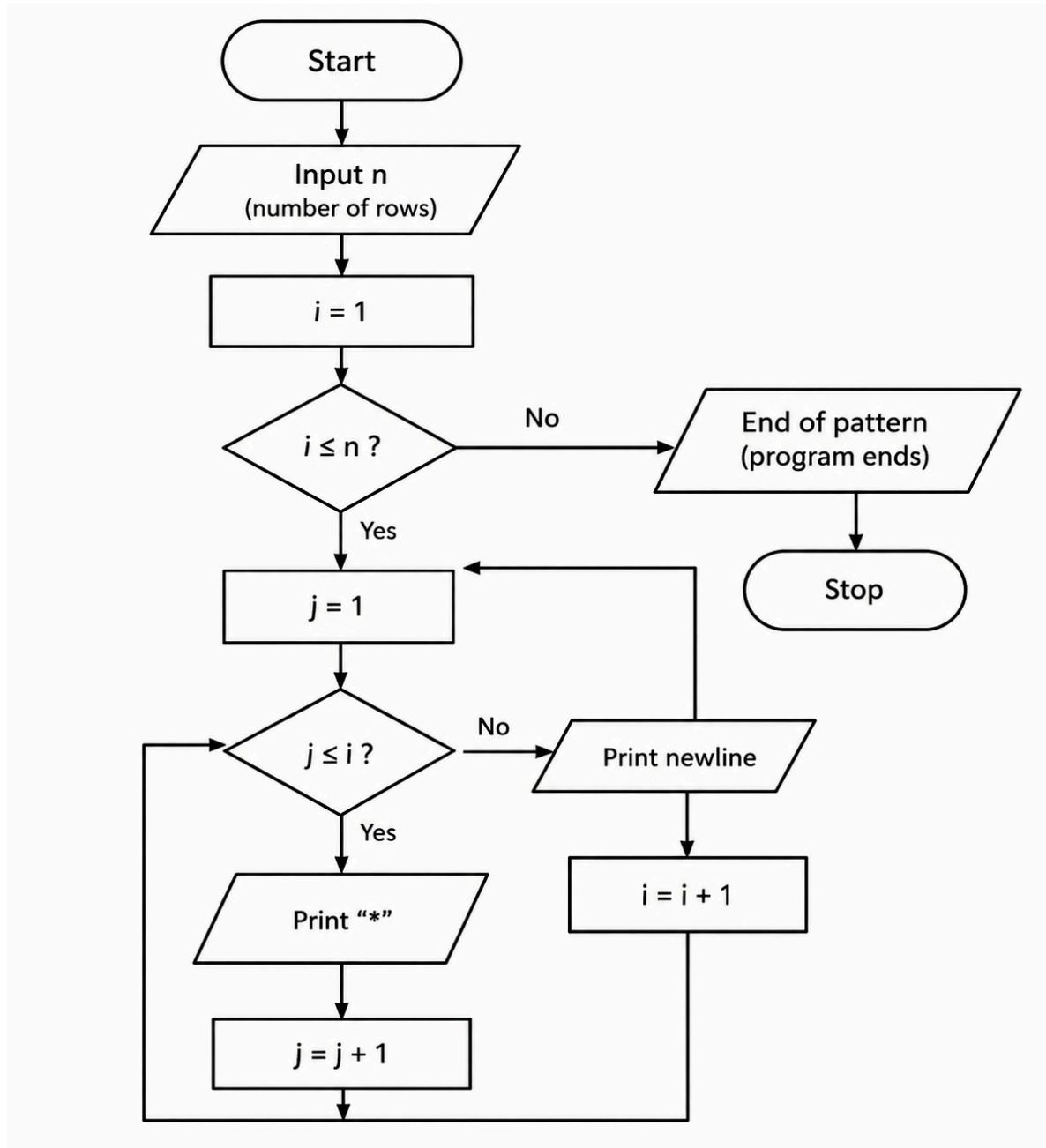
Conclusion : We successfully demonstrated displaying sum of digits using variables and while loop.

Q2. Write a C# program to print the following pattern:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Flow diagram / Logic:

An outer loop runs from $i = 1$ to 6, controlling the number of rows. An inner loop runs from $j = 1$ to less than i , printing the numbers 1 up to $(i-1)$ on each row, after which a newline is printed.



Program:

```
namespace TYIT_Rep01
{
    internal class Program
    {
        static void Main()
        {
            for (int i = 1; i <= 6; i++)
            {
                for (int j = 1; j < i; j++)
                {
                    Console.Write(" "+j+" ");
                }
                Console.WriteLine("\n");
            }
        }
    }
}
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

C:\Users\SXC_Student\...\TYIT_Rep01.exe (process 24532) exited with code 0 (0x0).
Press any key to close this window . . .
```

Conclusion : We successfully demonstrated displaying the pattern using variables and while loop.

Practical No : 02

Title : Introduction to C# Programming – Part 2

Date : 16/06/2026

Aim: To demonstrate use of multi-dimensional arrays in C#.

Q1. Write a C# code to find the diagonal and border sum of a matrix

Flow Diagram / Logic:

Two 3x3 matrices are read from the user. The dsum() function adds up elements where the row index equals the column index (the main diagonal). The bsum() function adds up all elements lying on the first/last row or first/last column (the border). The max_num() function scans every element to find the largest value.

Program:

```
using System;
using System.ComponentModel.DataAnnotations;

namespace Test_01
{
    internal class Program
    {
        public static int max_num(int[,] mat)
        {
            int max = 0;
            for (int i = 0; i < 3; i++)
            {
                for (int j = 0; j < 3; j++)
                {
                    if (mat[i, j] >= max)
                    {
                        max = mat[i, j];
                    }
                }
            }
            return max;
        }

        public static int dsum(int[,] mat)
        {
            int diagonalSum = 0;
            for (int i = 0; i < 3; i++)
            {
                diagonalSum += mat[i, i];
            }
            return diagonalSum;
        }

        public static int bsum(int[,] mat)
        {
            int sum = 0;
            int rows = mat.GetLength(0);
```

```

        int cols = mat.GetLength(1);
        for (int i = 0; i < rows; i++)
        {
            for (int j = 0; j < cols; j++)
            {
                if (i == 0 || i == rows - 1 || j == 0 || j == cols - 1)
                {
                    sum += mat[i, j];
                }
            }
        }
        return sum;
    }

    static void Main(string[] args)
    {
        int[,] mat1 = new int[3, 3];
        int[,] mat2 = new int[3, 3];
        Console.WriteLine("Enter Matrix 1:");
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                Console.Write($"Enter element [{i},{j}]: ");
                mat1[i, j] = int.Parse(Console.ReadLine());
            }
        }
        Console.WriteLine("Diagonal Sum of Matrix 1 = " + dsum(mat1));
        Console.WriteLine("max of Matrix 2 = " + max_num(mat1));

        Console.WriteLine("\nEnter Matrix 2:");
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                Console.Write($"Enter element [{i},{j}]: ");
                mat2[i, j] = int.Parse(Console.ReadLine());
            }
        }
        Console.WriteLine("Diagonal Sum of Matrix 2 = " + dsum(mat2));
        Console.WriteLine("Border Sum of Matrix 2 = " + bsum(mat2));
    }
}

```

Output:

```

Enter Matrix 1:
Enter element [0,0]: 8   Enter element [0,1]: 9   Enter element [0,2]: 4
Enter element [1,0]: 5   Enter element [1,1]: 4   Enter element [1,2]: 0
Enter element [2,0]: 7   Enter element [2,1]: 5   Enter element [2,2]: 9
Diagonal Sum of Matrix 1 = 21
max of Matrix 2 = 9

Enter Matrix 2: ... (values entered)
Diagonal Sum of Matrix 2 = 18
Border Sum of Matrix 2 = 44

```

Practical No : 03

Q1. Given a XML file of Product details. Write a C# code that fetches data from the XML file.

Title : Basic LINQ implementation in C#

Date : 17/06/2026

Aim: To demonstrate implementation of LINQ technology using XML

- (a) Display all products
- (b) Display name and price of products which have price > 55000
- (c) Display name and price of products which have price < 65000
- (d) Display name and price of products which have price < 55000 but <70000
- (e) Display name and price of products which are not 'DELL'
- (f) Display name and price of products which greater than 50,000 and not iphone
- (g) Count how many HP products are there.

Flow Diagram / Logic:

An XDocument loads productdata.xml. LINQ-to-XML queries then use Where/Select (and query-syntax 'from...where...select') on the Product elements, filtering by price, name, or manufacture date, and projecting the results into anonymous objects for display.

File: productdata.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<Products>
  <Product>
    <pid>101</pid>
    <pname>Samsung Refridgerator</pname>
    <price>200000</price>
    <dom>10-01-2026</dom>
  </Product>
  <Product>
    <pid>102</pid>
    <pname>Samsung Galaxy M78</pname>
    <price>560000</price>
    <dom>09-07-2026</dom>
  </Product>
  <Product>
    <pid>103</pid>
    <pname>Iphone</pname>
    <price>100000</price>
    <dom>10-10-2026</dom>
  </Product>
  <Product>
    <pid>104</pid>
    <pname>DELL Inspiron 15</pname>
    <price>55000</price>
    <dom>05-01-2026</dom>
  </Product>
</Products>
```

```

<Product>
  <pid>105</pid>
  <pname>Asus VivoBook 15</pname>
  <price>59000</price>
  <dom>10-05-2026</dom>
</Product>
<Product>
  <pid>106</pid>
  <pname>Apple MacBook Air M3</pname>
  <price>125000</price>
  <dom>15-06-2026</dom>
</Product>
<Product>
  <pid>107</pid>
  <pname>Microsoft Surface Laptop 6</pname>
  <price>138000</price>
  <dom>20-07-2026</dom>
</Product>
<Product>
  <pid>108</pid>
  <pname>Samsung Galaxy Book4</pname>
  <price>82000</price>
  <dom>05-08-2026</dom>
</Product>
<Product>
  <pid>109</pid>
  <pname>MSI Modern 14</pname>
  <price>67000</price>
  <dom>12-09-2026</dom>
</Product>
<Product>
  <pid>110</pid>
  <pname>LG Gram 16</pname>
  <price>145000</price>
  <dom>10-10-2026</dom>
</Product>
</Products>

```

File: Program.cs

```

using System.Linq;
using System.Linq.Expressions;
using System.Xml.Linq;

namespace xml_demo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            XDocument pd =
XDocument.Load("C:\\Users\\SXC_Student\\source\\repos\\xml_demo\\productdata.xml");
            var product = pd.Elements();

            Console.WriteLine("----- Display All Products -----");
            foreach (var item in product)
            {

```

```

        Console.WriteLine(item.ToString());
    }

    Console.WriteLine("----- Display Name and Price Of Products having
price > 55000");
    var result = product.Elements("Product").Where(x => (int)x.Element("price") >
55000).Select(x =>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,
            dom = x.Element("dom").Value
        }
    );
    foreach (var item in result) { Console.WriteLine(item.ToString()); }

    Console.WriteLine("----- Display Name and Price Of Products having
price < 65000");
    var result1 = product.Elements("Product").Where(x => (int)x.Element("price") <
65000).Select(x =>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,
            dom = x.Element("dom").Value
        }
    );
    foreach (var item in result1) { Console.WriteLine(item.ToString()); }

    Console.WriteLine("----- Display Name and Price Of Products having
price < 55000 but <70000");
    var result2 = product.Elements("Product").Where(x => (int)x.Element("price") >
55000 && (int)x.Element("price") < 70000).Select(x =>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,
            dom = x.Element("dom").Value
        }
    );
    foreach (var item in result2) { Console.WriteLine(item.ToString()); }

    Console.WriteLine("----- Display name and price of products which are
not 'DELL'");
    var result3 = product.Elements("Product").Where(x => (string)x.Element("pname") !=
"DELL").Select(x =>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,
            dom = x.Element("dom").Value
        }
    );
    foreach (var item in result3) { Console.WriteLine(item.ToString()); }

```

```

        Console.WriteLine("----- Display name and price of products which
greater than 50,000 and not iphone");
        var result4 = product.Elements("Product").Where(x => (int)x.Element("price") >
50000 && (string)x.Element("pname") != "Iphone").Select(x =>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,
            dom = x.Element("dom").Value
        }
    );
    foreach (var item in result4) { Console.WriteLine(item.ToString()); }

    Console.WriteLine("----- Count how many HP products are there
-----");
    int count = 0;
    var result5 = product.Elements("Product").GroupBy(x => (string)x.Element("pname")
== "%HP%");
    foreach (var item in result5) { count++; }
    Console.WriteLine(count);
}
}
}

```

Output:

(a) Display All Products

```

<Products>
  <Product><pid>101</pid><pname>Samsung
Refridgerator</pname><price>200000</price><dom>10-01-2026</dom></Product>
  <Product><pid>102</pid><pname>Samsung Galaxy
M78</pname><price>560000</price><dom>09-07-2026</dom></Product>
  ... (all 10 products listed)
</Products>

```

(b) Products with price > 55000

```

{ Name = Samsung Refridgerator, Price = 200000, dom = 10-01-2026 }
{ Name = Samsung Galaxy M78, Price = 560000, dom = 09-07-2026 }
{ Name = Iphone, Price = 100000, dom = 10-10-2026 }
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
{ Name = Apple MacBook Air M3, Price = 125000, dom = 15-06-2026 }
{ Name = Microsoft Surface Laptop 6, Price = 138000, dom = 20-07-2026 }
{ Name = Samsung Galaxy Book4, Price = 82000, dom = 05-08-2026 }
{ Name = MSI Modern 14, Price = 67000, dom = 12-09-2026 }
{ Name = LG Gram 16, Price = 145000, dom = 10-10-2026 }

```

(c) Products with price < 65000

```

{ Name = DELL Inspiron 15, Price = 55000, dom = 05-01-2026 }
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }

```

(d) Products with price > 55000 but < 70000

```

{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
{ Name = MSI Modern 14, Price = 67000, dom = 12-09-2026 }

```

(e) Products which are not 'DELL'

```

{ Name = Samsung Refridgerator, Price = 200000, dom = 10-01-2026 }
{ Name = Samsung Galaxy M78, Price = 560000, dom = 09-07-2026 }
{ Name = Iphone, Price = 100000, dom = 10-10-2026 }

```

```
{ Name = DELL Inspiron 15, Price = 55000, dom = 05-01-2026 }
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
{ Name = Apple MacBook Air M3, Price = 125000, dom = 15-06-2026 }
{ Name = Microsoft Surface Laptop 6, Price = 138000, dom = 20-07-2026 }
{ Name = Samsung Galaxy Book4, Price = 82000, dom = 05-08-2026 }
{ Name = MSI Modern 14, Price = 67000, dom = 12-09-2026 }
{ Name = LG Gram 16, Price = 145000, dom = 10-10-2026 }
(Note: the string match is exact, so 'DELL Inspiron 15' is not excluded here.)
```

(f) Products greater than 50,000 and not iPhone

```
{ Name = Samsung Refridgerator, Price = 200000, dom = 10-01-2026 }
{ Name = Samsung Galaxy M78, Price = 560000, dom = 09-07-2026 }
{ Name = DELL Inspiron 15, Price = 55000, dom = 05-01-2026 }
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
{ Name = Apple MacBook Air M3, Price = 125000, dom = 15-06-2026 }
{ Name = Microsoft Surface Laptop 6, Price = 138000, dom = 20-07-2026 }
{ Name = Samsung Galaxy Book4, Price = 82000, dom = 05-08-2026 }
{ Name = MSI Modern 14, Price = 67000, dom = 12-09-2026 }
{ Name = LG Gram 16, Price = 145000, dom = 10-10-2026 }
```

(g) Count how many HP products are there

```
----- Count how many HP products are there -----
1
*****
```

Error and Debug : None

Conclusion : We successfully demonstrated basic implementation of LINQ technology via integrating a XML local database

Practical No : 04

Title : Date and Time in LINQ implementation in C#

Date : 25/06/2026

Aim: To demonstrate implementation of LINQ technology using XML (fetching date and time fields)

Q1. Given a XML file of Product details. Write a C# code that fetches data from the XML file.

- (a) Show all product greater than 10-01-2026 menu date
- (b) Show all products where date is greater than 01-07-2026 and price is greater Rs 40000.
- (c) Show all products between date 01-07-2026 and 01-10-2026.

Flow Diagram / Logic:

The same productdata.xml is reused. Query-syntax LINQ (from...where...select) parses each product's 'dom' string into a DateTime using DateTime.ParseExact with format "dd-MM-yyyy", then compares it against reference dates to filter products, optionally combined with a price condition.

File: productdata.xml

(Same as Practical 03 — see above.)

File: Program.cs (relevant section)

```
var pr = from x in product.Elements("Product")
        where DateTime.ParseExact((string)x.Element("dom"), "dd-MM-yyyy", null) >
               DateTime.ParseExact("10-01-2026", "dd-MM-yyyy", null)
        let nm = (string)x.Element("pname")
        select nm;
Console.WriteLine("*****");
Console.WriteLine("Q. Show all product greater than 10-01-2026 manu date");
foreach (var item in pr) { Console.WriteLine(item); }
Console.WriteLine("*****");

Console.WriteLine("Q. 5");
var pr2 = from x in product.Elements("Product")
        where ((int)x.Element("price") > 40000) &&
               DateTime.ParseExact((string)x.Element("dom"), "dd-MM-yyyy", null)
               >
               DateTime.ParseExact("01-07-2026", "dd-MM-yyyy", null)
        select new
        {
            pname = (string)x.Element("pname"),
            price = (int)x.Element("price"),
            dom = (string)x.Element("dom")
        };
foreach (var item in pr2) { Console.WriteLine(item); }
Console.WriteLine("*****");

Console.WriteLine("Q. 6 ");
var pr3 = from x in product.Elements("Product")
        where DateTime.ParseExact((string)x.Element("dom"), "dd-MM-yyyy", null)
               >
```

```

        DateTime.ParseExact("01-07-2026", "dd-MM-yyyy", null)
        && DateTime.ParseExact((string)x.Element("dom"), "dd-MM-yyyy",
null) <
        DateTime.ParseExact("01-10-2026", "dd-MM-yyyy", null)
select new
{
    pname = (string)x.Element("pname"),
    price = (int)x.Element("price"),
    dom = (string)x.Element("dom")
};
foreach (var item in pr3) { Console.WriteLine(item); }
Console.WriteLine("*****");

```

Output:

(a) Products with dom greater than 10-01-2026

```

Q. Show all product greater than 10-01-2026 manu date
Samsung Galaxy M78
Iphone
Asus VivoBook 15
Apple MacBook Air M3
Microsoft Surface Laptop 6
Samsung Galaxy Book4
MSI Modern 14
LG Gram 16
*****

```

(b) Products with dom > 01-07-2026 and price > 40000

```

*****
Q. 5
{ pname = Samsung Galaxy M78, price = 560000, dom = 09-07-2026 }
{ pname = Iphone, price = 100000, dom = 10-10-2026 }
{ pname = Microsoft Surface Laptop 6, price = 138000, dom = 20-07-2026 }
{ pname = Samsung Galaxy Book4, price = 82000, dom = 05-08-2026 }
{ pname = MSI Modern 14, price = 67000, dom = 12-09-2026 }
{ pname = LG Gram 16, price = 145000, dom = 10-10-2026 }
*****

```

(c) Products with dom between 01-07-2026 and 01-10-2026

```

Q. 6
{ pname = Samsung Galaxy M78, price = 560000, dom = 09-07-2026 }
{ pname = Microsoft Surface Laptop 6, price = 138000, dom = 20-07-2026 }
{ pname = Samsung Galaxy Book4, price = 82000, dom = 05-08-2026 }
{ pname = MSI Modern 14, price = 67000, dom = 12-09-2026 }

```

Error and Debug : None

Conclusion : We successfully demonstrated basic implementation of LINQ technology via integrating a XML local database to fetch data based on date and time.

Practical No : 05

Title : Inheritance and hierarchy – Part 1

Date : 02/07/2026

Aim: To understand how constructor works

Q1. Write a C# code to demonstrate working of constructor and hierarchy.

Flow Diagram / Logic:

Class B inherits from class A, and Program inherits from B. When B's default constructor runs, it explicitly calls A's parameterised constructor via `base("trial")`, which in turn calls A's own default constructor via `this()`. This demonstrates constructor chaining across a three-level class hierarchy.

Program:

```
namespace Inheritance
{
    class A
    {
        public A()
        {
            Console.WriteLine("In A default");
        }
        public A(string n) : this()
        {
            Console.WriteLine("In a parameterised");
        }
    }

    class B : A
    {
        public B() : base("trial")
        {
            Console.WriteLine("In B default");
        }
        public B(string n)
        {
            Console.WriteLine("In B Parameterised");
        }
    }

    class Program : B
    {
        static void Main(string[] args)
        {
            Program t = new Program();
        }
    }
}
```

Output:

```
In A default
```

```
In a parameterised  
In B default  
  
C:\Users\SXC_Student\source\repos\Inheritance\bin\Debug\...  
Press any key to close this window . . .
```

Error and Debug : None

Conclusion : We successfully demonstrated how constructor works and types of constructor.

Practical No : 06

Title : Inheritance and hierarchy – Part 2

Date : 02/07/2026

Aim: To understand how inheritance works (override and virtual)

Q1. Write at-least four examples, demonstrating inheritance in C#.

Flow Diagram / Logic:

Each example builds a small class hierarchy (A, B, C, Program) with different combinations of virtual, override, and new modifiers on a Display() method, then calls Display() through a base-class reference to show whether C# resolves the call dynamically (via the virtual/override chain) or statically (via new, which hides rather than overrides).

Example 1:

```
namespace Inheritance
{
    class A
    {
        public virtual void Display()
        { Console.WriteLine("In A Display"); }
    }
    class B : A
    {
        public override void Display()
        { Console.WriteLine("In B Display"); }
    }
    class C : B
    {
        public override void Display()
        { Console.WriteLine("In C Display"); }
    }
    class Program : C
    {
        public override void Display()
        { Console.WriteLine("In Program Display"); }
        static void Main(string[] args)
        {
            A a = new Program();
            a.Display();
        }
    }
}
```

Output:

```
In Program Display
```

Example 2:

```
namespace Inheritance
{
    class A
```

```

    {
        public virtual void Display()
        { Console.WriteLine("In A Display"); }
    }
    class B : A
    {
        public new void Display()
        { Console.WriteLine("In B Display"); }
    }
    class Program
    {
        static void Main(string[] args)
        {
            A a = new B();
            a.Display();
        }
    }
}

```

Output:

```
In A Display
```

Example 3:

```

namespace Inheritance
{
    class A
    {
        public virtual void Display()
        { Console.WriteLine("In A Display"); }
    }
    class B : A
    {
        public new void Display()
        { Console.WriteLine("In B Display"); }
    }
    class Program
    {
        static void Main(string[] args)
        {
            A a = new B();
            a.Display();
        }
    }
}

```

Output:

```
In A Display
```

Example 4:

```

namespace Inheritance
{
    class A
    {
        public virtual void Display()

```

```

        { Console.WriteLine("In A Display"); }
    }
    class B : A
    {
        public override void Display()
        { Console.WriteLine("In B Display"); }
    }
    class C : B
    {
        public new void Display()
        { Console.WriteLine("In C Display"); }
    }
    class Program
    {
        static void Main(string[] args)
        {
            A a = new C();
            a.Display();
        }
    }
}

```

Output:

```
In B Display
```

Example 5:

```

namespace Inheritance
{
    class A
    {
        public virtual void Display()
        { Console.WriteLine("In A Display"); }
    }
    class B : A
    {
        public new virtual void Display()
        { Console.WriteLine("In B Display"); }
    }
    class C : B
    {
        public override void Display()
        { Console.WriteLine("In C Display"); }
    }
    class Program
    {
        static void Main(string[] args)
        {
            A a = new C();
            a.Display();
        }
    }
}

```

Output:

```
In A Display
```

Practical No : 07

Title : Structures in C#

Date : 09/07/2026

Aim: To demonstrate implementation of Structure in C#

Q1. Write a C# Program to store and retrieve student data. Take the following attributes:

- Roll No
- Name
- Age
- Course
- Marks

Flow Diagram / Logic:

A Student class holds roll number, name, age, course, and marks. An array of 3 Student objects is created; a loop reads each student's details from the console and constructs a Student object, and a second loop prints all stored student records.

Program:

```
namespace DataStructure
{
    class Student
    {
        public int rollno;
        public string name;
        public int age;
        public string course;
        public int marks;

        public Student(int roll, string name, int age, string course, int marks)
        {
            this.rollno = roll;
            this.name = name;
            this.age = age;
            this.course = course;
            this.marks = marks;
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            Student[] s = new Student[3];
            int roll;
            string name;
            int age;
            string course;
            int marks;

            for (int i = 0; i < s.Length; i++)
```

```

    {
        Console.WriteLine("Enter Record of Student :" + (i + 1));
        Console.WriteLine("Enter Roll No : ");
        roll = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Enter Name : ");
        name = Console.ReadLine();
        Console.WriteLine("Enter Course : ");
        course = Console.ReadLine();
        Console.WriteLine("Enter Age : ");
        age = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Enter Marks : ");
        marks = Convert.ToInt32(Console.ReadLine());

        s[i] = new Student(roll, name, age, course, marks);
    }

    for (int i = 0; i < s.Length; i++)
    {
        Console.WriteLine("-----");
        Console.WriteLine("Student " + (i + 1) + "info :");
        Console.WriteLine(s[i].name);
        Console.WriteLine(s[i].rollno);
        Console.WriteLine(s[i].age);
        Console.WriteLine(s[i].course);
        Console.WriteLine(s[i].marks);
        Console.WriteLine("-----");
    }
}
}
}

```

Output:

```

Enter Record of Student :1
Enter Roll No : 1   Enter Name : Ian Reddy   Enter Course : Bsc IT   Enter Age : 19   Enter Marks : 99
Enter Record of Student :2
Enter Roll No : 2   Enter Name : Adrian Dias   Enter Course : BAMCJ   Enter Age : 19   Enter Marks : 90
Enter Record of Student :3
Enter Roll No : 3   Enter Name : Shreyas Bhandary   Enter Course : Bsc IT   Enter Age : 19   Enter Marks : 78
-----
Student 1info :
Ian Reddy
1
19
Bsc IT
99
-----
Student 2info :
Adrian Dias
2
19
BAMCJ
90
-----

```

```
Student 3info :  
Shreyas Bhandary  
3  
19  
Bsc IT  
78  
-----
```

Q2. Write a C# Program to store and retrieve employee data. Take the following attributes:

- Employee Number (eno)
- Name
- Age
- Salary

Retrieve:

- (a) Name and age of employees whose salary is greater than 60000.
- (b) Employees who are above 50 in age.
- (c) Display the age of employee who has maximum salary.

Program:

```
using System.ComponentModel.DataAnnotations;  
using System.Globalization;  
  
namespace DataStructure  
{  
    class emp  
    {  
        public int eno;  
        public string name;  
        public int age;  
        public int salary;  
  
        public emp(int eno, string name, int age, int salary)  
        {  
            this.eno = eno;  
            this.name = name;  
            this.age = age;  
            this.salary = salary;  
        }  
    }  
  
    internal class Program  
    {  
        static void Main(string[] args)  
        {  
            emp[] s = new emp[5];  
            int eno;  
            string name;  
            int age;  
            int salary;
```

```

for (int i = 0; i < s.Length; i++)
{
    Console.WriteLine("Enter Record of Employee :" + (i + 1));
    Console.WriteLine("Enter Employee Id : ");
    eno = Convert.ToInt32(Console.ReadLine());
    Console.WriteLine("Enter Name : ");
    name = Console.ReadLine();
    Console.WriteLine("Enter Age : ");
    age = Convert.ToInt32(Console.ReadLine());
    Console.WriteLine("Enter Salary : ");
    salary = Convert.ToInt32(Console.ReadLine());

    s[i] = new emp(eno, name, age, salary);
}

// Display name and age of employee whose salary > 60000
for (int i = 0; i < s.Length; i++)
{
    if (s[i].salary > 60000)
    {
        Console.WriteLine("Name : " + s[i].name);
        Console.WriteLine("Age : " + s[i].age);
    }
}

// Display the age of emp who all are > 50
for (int i = 0; i < s.Length; i++)
{
    if (s[i].age > 50)
    {
        Console.WriteLine("Name : " + s[i].name);
        Console.WriteLine("Age : " + s[i].age);
    }
}

// Display the age of emp who has max salary
int max = 0;
string max_namo = null;
for (int i = 0; i < s.Length; i++)
{
    if (s[i].salary > max)
    {
        max = s[i].salary;
        max_namo = s[i].name;
    }
}
Console.WriteLine("MAX SALARY BY : " + max_namo + " " + max);
}
}
}

```

Output:

```

Enter Name : Joy   Enter Age : 80   Enter Salary : 67000
Display name and age of employee whose salary > 60000
-----
Name : Saksham Dias   Age : 19

```

```
Name : Adrian   Age : 21
Name : Joy     Age : 80
Display the age of emp who all are > 50
-----
Name : Joy     Age : 80
Display the age of emp who has max salary
-----
MAX SALARY BY : Saksham Dias 98000
```

Conclusion: We successfully implemented structure to store and retrieve student and employee details.

Practical No : 08

Title : ASP.NET Razor Pages

Date : 09/07/2026

Aim: To demonstrate implementation of Razor pages.

Q1. Create a website that manages CRUD operation for Employees using Razor Pages MVC.

Flow Diagram / Logic:

An ASP.NET Core Razor Pages application is created with an Employee model, an in-memory (or Entity Framework Core) data context, and a set of Razor Pages under /Pages/Employees/ — Index (list), Create (add), Edit (update), Delete (remove), and Details (view one record). Each .cshtml page has a matching PageModel (.cshtml.cs) that handles OnGet and OnPost requests, binding form data to the Employee model and persisting changes through the DbContext.

Model: Employee.cs

```
namespace RazorCRUD.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public int Age { get; set; }
        public decimal Salary { get; set; }
    }
}
```

Data Context: EmployeeContext.cs

```
using Microsoft.EntityFrameworkCore;
using RazorCRUD.Models;

namespace RazorCRUD.Data
{
    public class EmployeeContext : DbContext
    {
        public EmployeeContext(DbContextOptions<EmployeeContext> options) : base(options) { }
        public DbSet<Employee> Employees { get; set; }
    }
}
```

Pages/Employees/Index.cshtml.cs (Read - list all employees)

```
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorCRUD.Data;
using RazorCRUD.Models;

namespace RazorCRUD.Pages.Employees
{
```

```

public class IndexModel : PageModel
{
    private readonly EmployeeContext _context;
    public IndexModel(EmployeeContext context) { _context = context; }
    public IList<Employee> Employee { get; set; } = default!;

    public async Task OnGetAsync()
    {
        Employee = await _context.Employees.ToListAsync();
    }
}

```

Pages/Employees/Create.cshtml.cs (Create)

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using RazorCRUD.Data;
using RazorCRUD.Models;

namespace RazorCRUD.Pages.Employees
{
    public class CreateModel : PageModel
    {
        private readonly EmployeeContext _context;
        public CreateModel(EmployeeContext context) { _context = context; }

        [BindProperty]
        public Employee Employee { get; set; } = default!;

        public IActionResult OnGet() { return Page(); }

        public async Task<IActionResult> OnPostAsync()
        {
            if (!ModelState.IsValid) return Page();
            _context.Employees.Add(Employee);
            await _context.SaveChangesAsync();
            return RedirectToPage("./Index");
        }
    }
}

```

Pages/Employees/Edit.cshtml.cs (Update)

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorCRUD.Data;
using RazorCRUD.Models;

namespace RazorCRUD.Pages.Employees
{
    public class EditModel : PageModel
    {
        private readonly EmployeeContext _context;
        public EditModel(EmployeeContext context) { _context = context; }
    }
}

```

```

[BindProperty]
public Employee Employee { get; set; } = default!;

public async Task<IActionResult> OnGetAsync(int id)
{
    var employee = await _context.Employees.FindAsync(id);
    if (employee == null) return NotFound();
    Employee = employee;
    return Page();
}

public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid) return Page();
    _context.Attach(Employee).State = EntityState.Modified;
    await _context.SaveChangesAsync();
    return RedirectToPage("./Index");
}
}
}

```

Pages/Employees/Delete.cshtml.cs (Delete)

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorCRUD.Data;
using RazorCRUD.Models;

namespace RazorCRUD.Pages.Employees
{
    public class DeleteModel : PageModel
    {
        private readonly EmployeeContext _context;
        public DeleteModel(EmployeeContext context) { _context = context; }

        [BindProperty]
        public Employee Employee { get; set; } = default!;

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var employee = await _context.Employees.FirstOrDefaultAsync(m => m.Id == id);
            if (employee == null) return NotFound();
            Employee = employee;
            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            var employee = await _context.Employees.FindAsync(id);
            if (employee != null)
            {
                _context.Employees.Remove(employee);
                await _context.SaveChangesAsync();
            }
            return RedirectToPage("./Index");
        }
    }
}

```

```
}  
}
```

Pages/Employees/Index.cshtml (excerpt)

```
@page  
@model RazorCRUD.Pages.Employees.IndexModel  
  
<h2>Employee List</h2>  
<a asp-page="Create">+ Add New Employee</a>  
<table>  
  <tr><th>Name</th><th>Department</th><th>Age</th><th>Salary</th><th>Actions</th></tr>  
  @foreach (var item in Model.Employee)  
  {  
    <tr>  
      <td>@item.Name</td>  
      <td>@item.Department</td>  
      <td>@item.Age</td>  
      <td>@item.Salary</td>  
      <td>  
        <a asp-page="./Edit" asp-route-id="@item.Id">Edit</a> |  
        <a asp-page="./Delete" asp-route-id="@item.Id">Delete</a>  
      </td>  
    </tr>  
  }  
</table>
```

Output:

Running the application opens the Employee Index page in the browser, listing all employee records with Edit and Delete links, and a link to add a new employee.

```
Employee List  
+ Add New Employee
```

Name	Department	Age	Salary	Actions
Riya Shah	HR	28	45000	Edit Delete
Karan Mehta	IT	32	72000	Edit Delete

Clicking '+ Add New Employee' opens the Create form; submitting it saves a new record and redirects back to this list. Edit opens a pre-filled form for the selected employee; Delete asks for confirmation before removing a record.

Error and Debug : None

Conclusion : We successfully created a Razor Pages MVC website implementing full CRUD (Create, Read, Update, Delete) operations for Employee records.